

ECE 220 Computer Systems & Programming

Lecture 11 — Problem Solving with Pointers and Arrays

September 29, 2026

Prof. Sayan Mitra mitras@illinois.edu Office hours: TBD
Course website: courses.grainger.illinois.edu/ece220/fa2026

 **ILLINOIS**
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

The Finding Problem

**You own the tools now — pointers, arrays, functions.
Time to solve the oldest problems: find it, and sort it.**



Sorted data turns a thousand probes into ten.

Today's Two Questions

Every probe should kill half the possibilities.

1. How fast can you *find* something in an array — and what does sortedness buy you?
2. How do the classic sorts move data — and what does each pass accomplish?

Everything today is Midterm-1 material applied: loops, functions, pointers, arrays — working together.

Warm-up: Transpose, Flat

Transpose an $M \times N$ matrix into an $N \times M$ one — both stored **flat** (as `int *`).

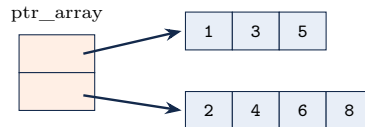
- `in[i][j]` is flat
`in_matrix[_____]`
- It must land at
`out_matrix[_____]`
- Why must the callee be told the shape?

```
void transpose(int *in_matrix, int *out_matrix) {  
    /* loop over rows and columns of the input;  
     */  
    /* in[i][j] lives at flat offset i*N + j --  
     */  
    /* where does it go in the N x M output?  
     */  
}
```

3×4 input 1..12 $\rightarrow$ first output row _____

Pointer Array vs. Pointer to an Array

```
int a[3] = {1,3,5};  
int b[4] = {2,4,6,8};  
  
int *ptr_array[2];    /* 2 pointers */  
ptr_array[0] = a;  
ptr_array[1] = b;
```



- `ptr_array[1][2] = _____`
- Rows may have *different* lengths — unlike `int m[2][4]`

Linear vs. Binary Search

Linear: check from the start until found — works on any array.

Binary (sorted arrays only): probe the middle, keep the half that can contain the item.

Step	Live region	Middle / action
search 23 in	2 5 8 12 16 23 38 56 72 91	
probe 1	indices 0–9	mid = _____
probe 2	indices 5–9	mid = _____
probe 3	indices 5–6	mid = _____

In-class: 1000 sorted elements — how many probes at most? _____

In-Class Problem: Iterative Binary Search

Return the item's index, or -1 .

- What do `low` and `high` mean?

- Loop while _____

- Why does it terminate?

```
int binary_search(int array[], int item) {  
    /* low and high bracket the live region */  
  
    /* while the region is non-empty:      */  
        /* find the middle; found? return */  
        /* else shrink to the correct half */  
  
    /* not found */  
}
```

Bubble Sort

One **pass**: compare each pair of neighbors, swap when out of order. Repeat passes until a pass makes _____.

5	1	4	2	8
1	5	4	2	8
1	4	5	2	8
1	4	2	5	8

(after pass 1 — 8 has “bubbled up”)

After pass k , the largest _____ elements are in place.

```
void bubble_sort(int array[]) {  
    /* declare counter and "swapped" flag */  
  
    do {  
        /* reset the flag */  
  
        /* one pass: compare neighbors, */  
        /* swap out-of-order pairs, */  
        /* remember that you swapped */  
  
    } while ( ); /* when to go again? */  
}
```

Note `swap(&array[i], &array[i+1])` — Lecture 9's pointer swap, earning its keep.

Insertion Sort

Like sorting cards in your hand:

Step	Array (sorted part in bold)
start: first item is “sorted”	5 2 6 1 3 9
take 2; shift 5 right; insert	_____
take 6; nothing to shift	_____
take 1; shift 6, 5, 2; insert	_____

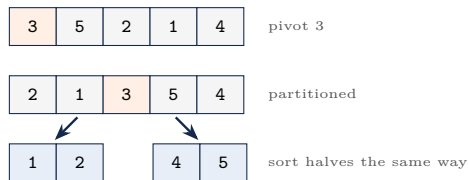
- Invariant: after step k , the first _____ elements are sorted (relative to each other)
- Great on nearly-sorted data — each item moves only as far as it must

Quick Sort (a Preview)

Divide and conquer:

1. Pick a **pivot**; **partition**: smaller elements left, larger right
2. The pivot is now in its final place
3. Sort each side *by the same method*

“Sort each side by the same method” —
quicksort calls *itself*. That idea gets a whole
lecture: recursion, after the midterm.



Summary & What's Next

Today

- **Flat matrices:** shape is yours to track — $i*N+j$ in, $j*M+i$ out
- **Searching:** linear works anywhere; binary needs sorted data and kills half the region per probe
- **Sorting:** bubble (neighbor swaps until a clean pass), insertion (grow a sorted prefix), quicksort (partition around a pivot — and call yourself)

Next

- **Midterm 1 is Thursday, October 1** — everything through today
- Next lecture (Oct 6): recursion — functions that call themselves

Code from today: `transpose.c`, `bsearch.c`, `bubble.c`.